Data Mining Assignment 2 Report

- Author: Juiyun Chang (314551145)
- Github Link: https://github.com/pinkypacman/DM2026-Assignment
- Repo Structure: All code are in `asg2/`
  - Problem 1's code is under Real_World_classification.ipynb.
  - results of questions 2~5 are in results.ipynb

1. K-fold Cross-Validation

1-a. K-fold Cross-Validation's optimal

- 5-fold CV Average Accuracy:

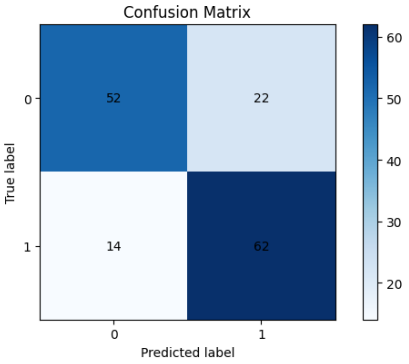
	lambda=1.0	**lambda=2.0**	**lambda=4.0**	**lambda=8.0**
lr=0.005	0.7171	0.7200	0.7257	0.7228
lr=0.01	0.7229	0.7286	0.7257	0.7171
lr=0.1	0.7229	0.7257	0.7257	0.7171
lr=0.5	0.7229	0.6914	0.5343	0.4657

- Top 2 Hyperparameter Settings:
  - Learning Rate: 0.01, reg_lambda: 2.0 (Accuracy: 0.7286, Std: 0.0579)
  - Learning Rate: 0.005, reg_lambda: 4.0 (Accuracy: 0.7257, Std: 0.0331)

1-b. K-fold Cross-Validation's optimal

- Rank 1 Model:

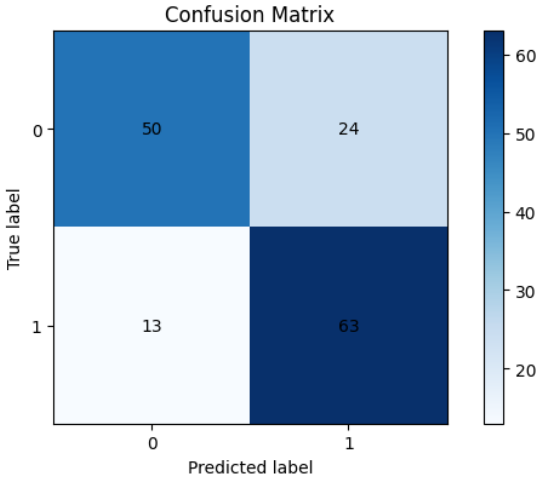
  lr=0.01, reg_lambda=2.0
  - Accuracy : 0.7600
  - Precision : 0.7381
  - Recall : 0.8158
  - F1-score : 0.7750



- Rank 2 Model:

  lr=0.005, reg_lambda=4.0

  - Accuracy : 0.7533
  - Precision : 0.7241
  - Recall : 0.8289
  - F1-score : 0.7730



1-c. Observations in (1-a) and (1-b)

- 1-a observations
  - Rank 1 (lr=0.01, λ=2.0, 0.7286) wins on mean CV accuracy. Rank 2 (lr=0.005, λ=4.0, 0.7257) is selected from the 0.7257 tie because its fold-std is roughly half (0.0331 vs 0.0579) — accuracy and stability disagree.
  - Stable learning rates (0.005–0.1) all sit in the 0.72–0.73 band; λ has only a small effect (≈±0.01). The λ=4.0 column is flat 0.7257 across all three stable learning rates — the most reproducible setting.
  - lr=0.5 is unstable and degrades sharply as λ grows (0.7229 → 0.4657), since the effective step size (lr × λ) causes oscillation/divergence.
  - Learning rate and λ must be tuned jointly, and the "best" setting depends on whether you optimize for peak accuracy or fold-robustness.
- 1-b observations
  - Cross Validataion(CV) ranking holds on test (0.7600 > 0.7533), matching the CV order. Test accuracy exceeds CV for both, so CV was conservative rather than overfit.
  - The two models are nearly interchangeable in F1 (0.7750 vs 0.7730) but differ in balance: Rank 1 leans precision (0.7381), Rank 2 leans recall (0.8289)
  - Rank 2's lower CV std is consistent with its smoother behavior on test (smaller , stronger λ).

2. SVM

2-a. Results of SVM classifier with C= 1.0

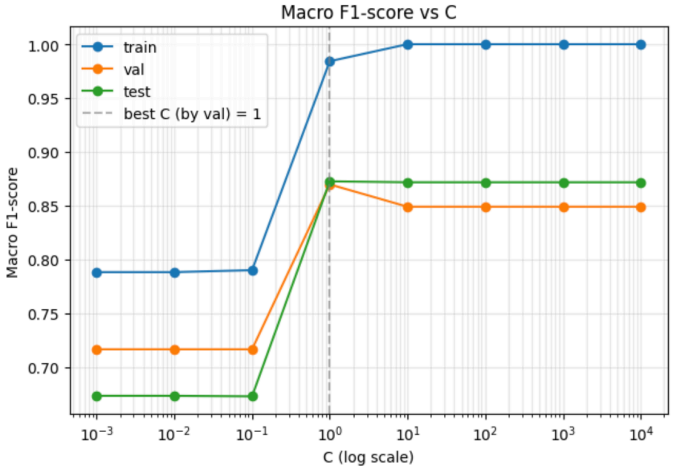
	accuracy	macro_f1
Train	0.9842	0.9842
Validation	0.8700	0.8698
Test	0.8725	0.8727

2-b. Sweep C and visualize how performance changes

- Marco F1 vs C

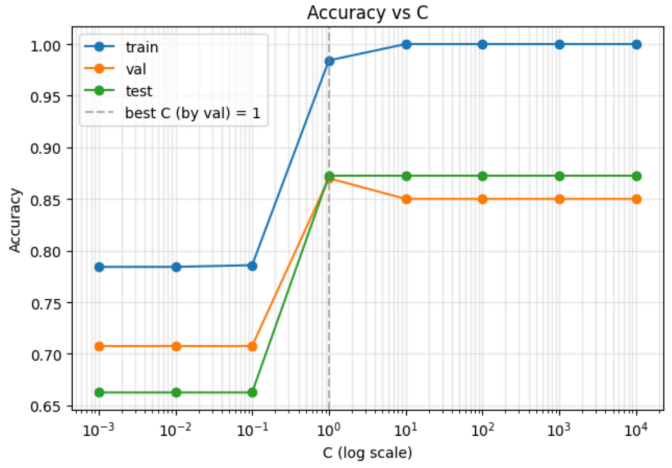
C	train	val	test
0.001	0.7884	0.7167	0.6735

C	train	val	test
0.01	0.7884	0.7167	0.6735
0.1	0.7902	0.7167	0.6730
1	0.9842	0.8698	0.8727
10	1.0000	0.8491	0.8718
100	1.0000	0.8491	0.8718
1000	1.0000	0.8491	0.8718
10000	1.0000	0.8491	0.8718



- Accuracy vs C

C	train	val	test
0.001	0.7842	0.7075	0.6625
0.01	0.7842	0.7075	0.6625
0.1	0.7858	0.7075	0.6625
1	0.9842	0.8700	0.8725
10	1.0000	0.8500	0.8725
100	1.0000	0.8500	0.8725
1000	1.0000	0.8500	0.8725
10000	1.0000	0.8500	0.8725



2-c. Best generalizing C and observations

	accuracy	macro_f1
Train	0.9842	0.9842
Validation	0.8700	0.8698
Test	0.8725	0.8727

I picked the C that maximizes validation macro-F1, see it as the best generalization performance.

- Observation
  - Underfitting region (`C ≤ 0.1`): All three curves are flat and low (train ≈ 0.79, val ≈ 0.71, test ≈ 0.66). Strong regularization shrinks the margin penalty so much that the model cannot fit the structure of the data — classic high-bias behavior.
  - Sharp transition at `C = 1`: A single decade jump in C lifts val/test by ~15–20 points and pushes train to ~0.98. The decision boundary suddenly has enough slack-penalty weight to separate the classes properly.
  - Saturation region (`C ≥ 10`): Train hits 1.00 and stays there, while val/test plateau slightly *below* their C=1 values (val drops from ~0.87 to ~0.85). This is mild overfitting — the model memorizes the training set but generalization stops improving.

3. Association Rule Mining

3-a. Frequent patterns with support ≥ 0.3

	support	Itemsets
0	0.682	[ram_medium]
1	0.416	[px_width_medium]
2	0.414	[battery_power_medium]
3	0.412	[int_memory_medium]
4	0.318	[battery_power_medium, ram_medium]
5	0.316	[int_memory_low]
6	0.308	[battery_power_low]
7	0.306	[px_width_medium, ram_medium]

3-b. Rules with support ≥ 0.3, confidence ≥ 0.4, lift ≥ 0.8

	antecedents	consequents	support	confidence	lift
0	[battery_power_medium]	[ram_medium]	0.318	0.7681	1.1263
1	[ram_medium]	[battery_power_medium]	0.318	0.4663	1.1263
2	[px_width_medium]	[ram_medium]	0.306	0.7356	1.0786
3	[ram_medium]	[px_width_medium]	0.306	0.4487	1.0786

3-c. Observations on (3a) & (3b)

Filtered to price_range == 1 *(medium-cost phones, 500 rows)*

- Highest-support single-item pattern**:** ram_medium *(support 0.682)*
  Meaning RAM in this class is overwhelmingly in the middle third of its global range.
- The other three features have their `*_medium` items at support around 0.41, with `*_low` items as the second most frequent.
- Among the rules, the strongest lift (~1.13) is for `battery_power_medium ↔ ram_medium:` phones in the medium price range tend to pair medium battery with medium RAM.

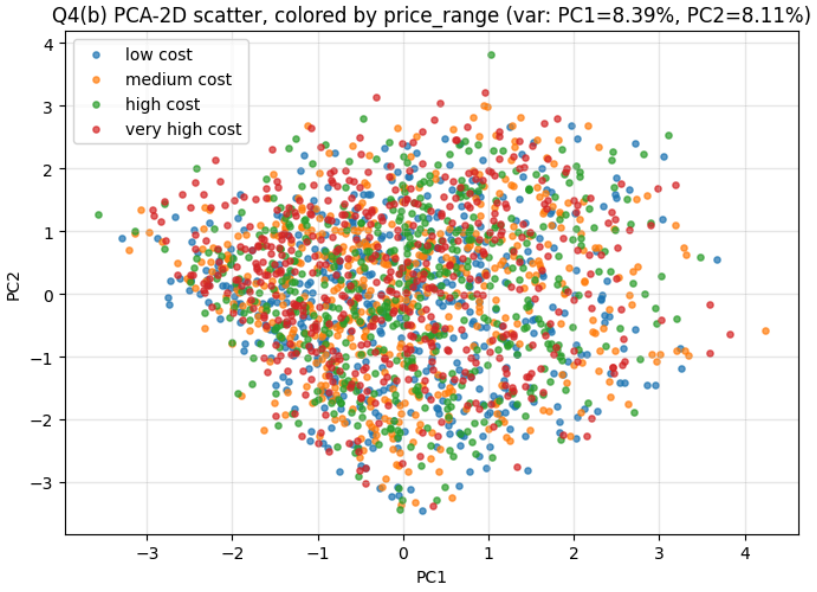
The reverse direction has lower confidence (0.466) because `ram_medium` is much more common than `battery_power_medium`, but the symmetric lift still indicates a real association rather than independence.

4. PCA and K-Means

4-a. Split data and standardize the features using z-score standardization.

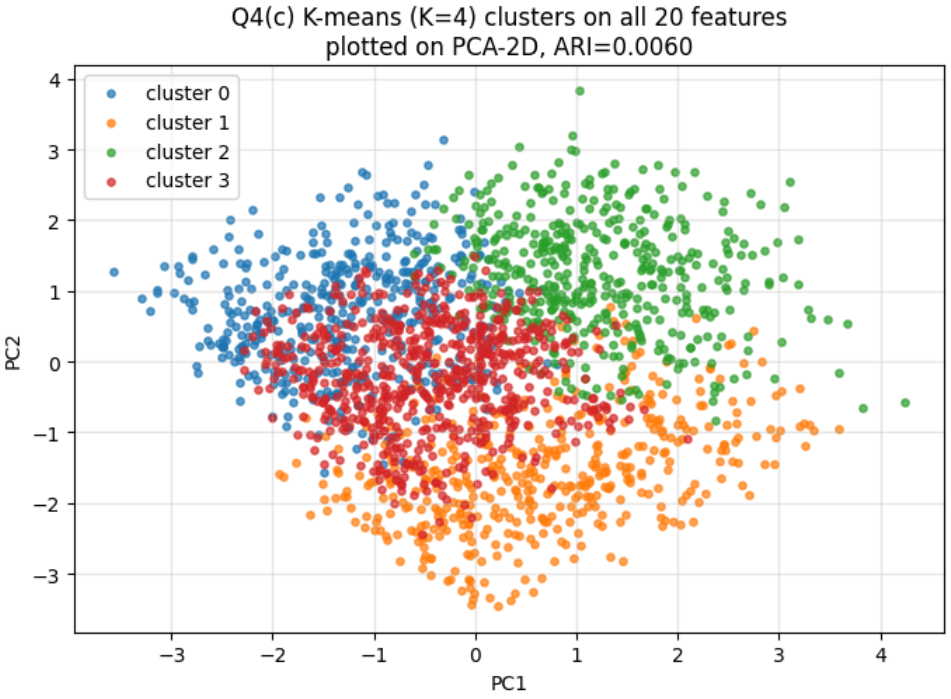
```python
def load_q4_data() -> Q4Data:
    df = pd.read_csv(DATA_PATH)
    y = df["price_range"].to_numpy()
    X = df.drop(columns=["price_range"]).to_numpy()
    Z = StandardScaler().fit_transform(X)
    pca = PCA(n_components=2, random_state=RANDOM_STATE).fit(Z)
    pca2 = pca.transform(Z)
    return Q4Data(X=X, Z=Z, pca2=pca2, pca=pca, y=y)
```

4-b. PCA-2D scatter



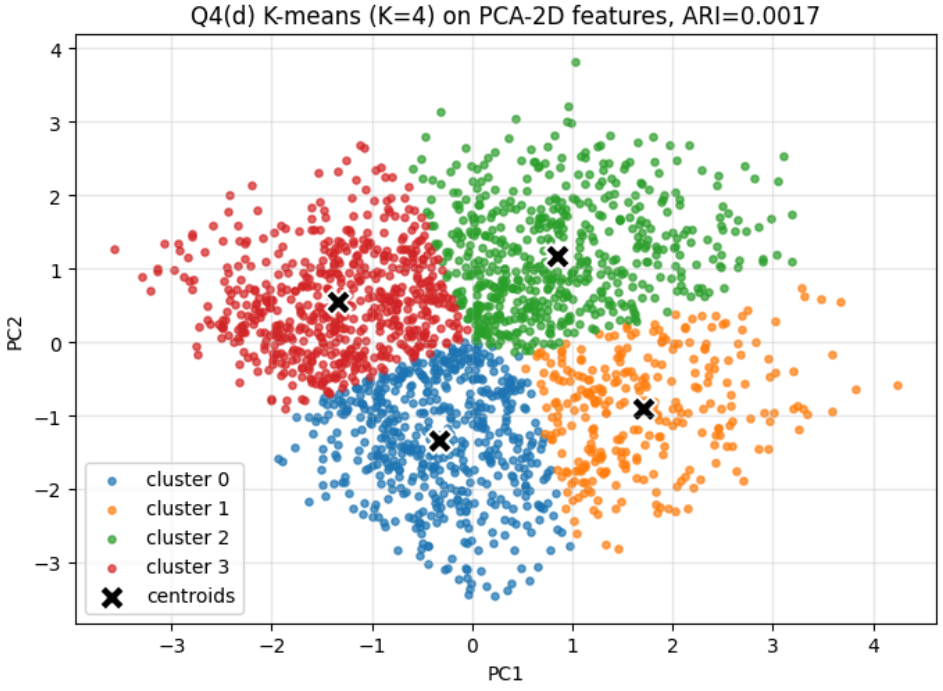
4-c. K-means (K=4) on all 20 standardized features, plotted in PCA-2D

- ARI = 0.006



4-d. K-means (K=4) on the 2-D PCA features

- ARI = 0.0017



4-e. observations in (4c) and (4d)

- **Two PCs are far from sufficient:**

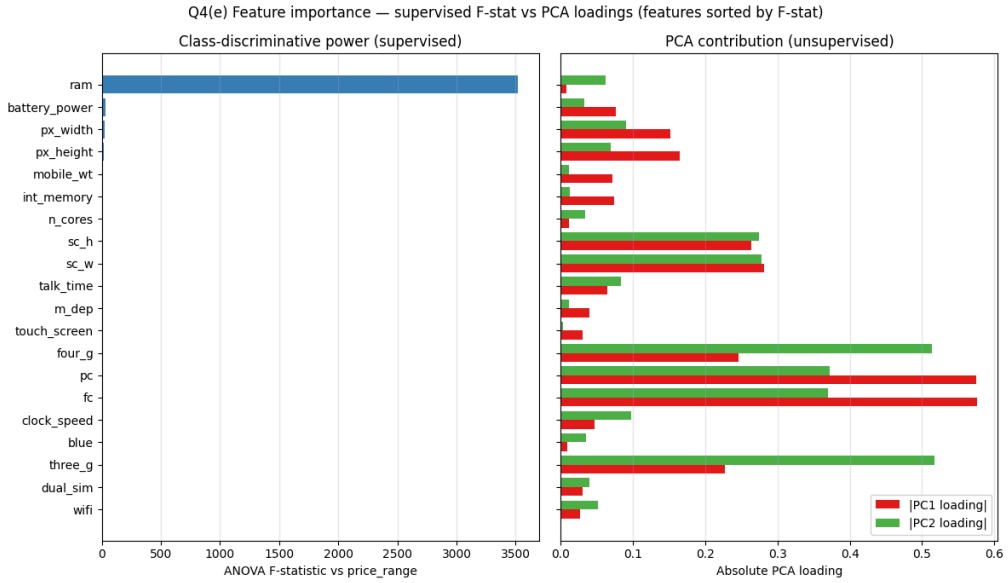
  PC1 and PC2 each explain only ~8% of the variance (~16% combined). Standardization gives every feature unit variance, so PCA has no high-variance direction to anchor on; loadings spread broadly across the 20 features.

- **K-means is correspondingly poor in both regimes:**

- On the full 20-D standardized space (4c), ARI = 0.006 — clusters do not align with classes.
- On the 2-D PCA projection (4d), ARI ≈ 0.002 — even worse.

- **The feature-importance figure above shows *why* (4d) is striking:**

  The supervised F-statistic is dominated by a single feature: `ram`, yet PCA's two components are dominated by the least class-relevant features:



- **The failure nodes are as follow:**
  - (4c) fails because, in the standardized 20-D space, `ram` is only one of 20 unit-variance dimensions; its discriminative signal is overwhelmed by 19 near-noise dimensions when computing isotropic Euclidean distances.
  - (4d) fails because the 2-D PCA projection discards the `ram` axis—the one direction that actually separates the classes—and instead projects onto noise dimensions like camera megapixels and 3G/4G flags.

5. Enhancing K-means with Association Rule Mining

5.1 Proposed method — Class-Conditional Rule Evidence (CCRE)

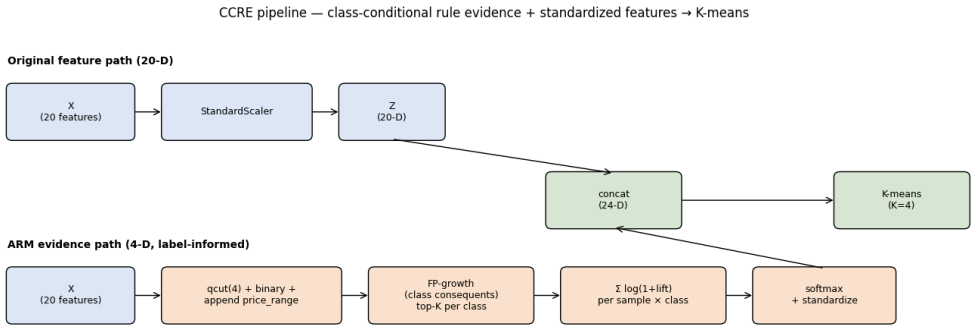
Q4 showed that vanilla K-means cannot recover `price_range` (ARI ≈ 0.006): the four classes are not Euclidean-spherical clusters in the 20-D standardized feature space. To achieve a meaningful improvement, we need to provide K-means with some label information; ARM is the vehicle.

Each sample is assigned a 4-dim soft class-membership feature derived from association rules whose consequent is `price_range_X`. K-means then runs on the concatenation of those 4 features with the 20 standardized originals (24-D total).

Inspirations:

- **Data observation (Q4(e)):** the supervised F-statistic is dominated by `ram`, but PCA's PC1/PC2 load almost entirely on class-irrelevant features (`fc`, `pc`, `three_g`). Any purely Euclidean clusterer is therefore looking in the wrong directions — we need a feature channel that explicitly encodes class-discriminative structure.
- **Class-association-rule classifiers (CBA, Liu et al. 1998):** rules of the form `antecedent → class` with high lift act as class-conditional evidence. Aggregating their lift per sample yields a class-membership score analogous to a log-likelihood ratio, which is exactly the signal K-means lacks.

This is a label-informed enhancement — `price_range` appears as a consequent during rule mining, so it should not be interpreted as purely unsupervised clustering.



Top 3 class-consequent rules per class, ranked by lift

class_label	antecedents	support	confidence	lift
0	[px_width_q1, ram_q1]	0.0590	0.9672	3.8689
0	[battery_power_q1, ram_q1]	0.0580	0.9667	3.8667
0	[battery_power_q2, ram_q1]	0.0635	0.9621	3.8485
1	[px_width_q2, ram_q2]	0.0510	0.7612	3.0448
1	[px_height_q3, ram_q2]	0.0540	0.7606	3.0423
1	[ram_q2, sc_w_q2]	0.0520	0.7591	3.0365
2	[ram_q3, three_g_1, touch_screen_0]	0.0675	0.7105	2.8421
2	[blue_0, ram_q3, three_g_1]	0.0710	0.7030	2.8119
2	[ram_q3, three_g_1, wifi_0]	0.0630	0.6923	2.7692
3	[battery_power_q4, ram_q4]	0.0590	0.9916	3.9664
3	[px_height_q4, ram_q4]	0.0630	0.9844	3.9375
3	[px_width_q4, ram_q4, three_g_1]	0.0515	0.9626	3.8505

Every single rule contains a `ram_qN` item, and the RAM quartile lines up perfectly with the price tier:

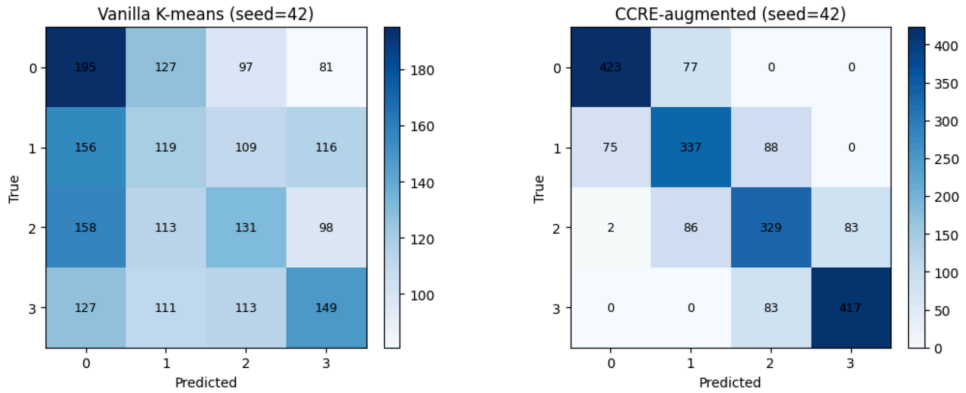
- `ram_q1` → class 0
- `ram_q2` → class 1
- `ram_q3` → class 2
- `ram_q4` → class 3

That's the dataset telling the RAM alone almost determines price. The other antecedents (px_width, battery, screen size, 3G) are minor refinements.

5.2 Vanilla K-means vs CCRE-augmented K-means results

- Results are averaged over 5 seeds

method	accuracy	precision	recall	f1
vanilla	0.2962 ± 0.0016	0.2959 ± 0.0017	0.2962 ± 0.0016	0.2941 ± 0.0012
CCRE (our enhancement)	0.7530 ± 0.0000	0.7530 ± 0.0000	0.7530 ± 0.0000	0.7530 ± 0.0000



5.3 Observations on the CCRE enhancement

- Vanilla K-means is at chance

  Across 5 seeds it sits at F1 ≈ 0.294, barely above the 0.25 chance level for a balanced 4-class problem.

  (4c) already explained why: ARI ≈ 0.006 says price_range is not recoverable from Euclidean geometry on the 20 standardized features, so no amount of seed luck will rescue it.

- CCRE reaches F1 ≈ 0.75 (accuracy / precision / recall all ≈ 0.75 too).

  The seed-to-seed std is 0.0000 — once the four evidence columns are added, K-means converges to the same partition regardless of initialization, which is the strongest possible signal that the evidence channel dominates the clustering.

- The signal source is interpretable

  The class-rule table shows lift ≈ 3.9 for `ram_q1 → class 0` and `ram_q4 → class 3`. This matches Q4(e)'s F-stat ranking: `ram` is overwhelmingly the most class-discriminative feature, and FP-growth surfaces it as the dominant antecedent across all four classes.

5.4 Ablation — Which design step matters most?

- Variants in the ablation study

Variant	Features fed to K-means	What's tested
vanilla	Z only (20-D)	baseline
+evidence	Z • raw Σ log(1+lift) per class (24-D)	skip softmax and standardize
+softmax	Z + softmax probabilities (24-D)	skip standardize only
CCRE (our proposed improvement)	Z + standardized softmax (24-D)	full pipeline
evidence only	standardized softmax alone (4-D)	drop original features

  - Z is the standardized 20 original features

- Results

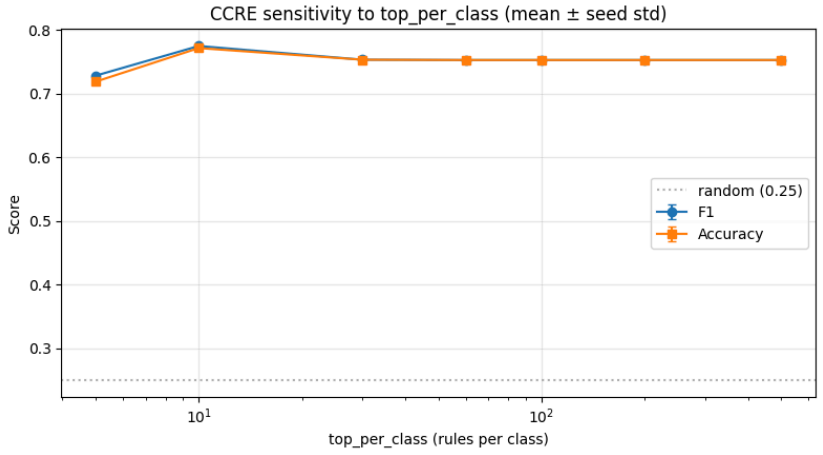
variant	accuracy	precision	recall	f1
vanilla	0.2962 ± 0.0016	0.2959 ± 0.0017	0.2962 ± 0.0016	0.2941 ± 0.0012
Z + raw evidence	0.7525 ± 0.0000	0.7546 ± 0.0000	0.7525 ± 0.0000	0.7534 ± 0.0000
Z + softmax	0.2992 ± 0.0019	0.2984 ± 0.0019	0.2992 ± 0.0019	0.2968 ± 0.0015

variant	accuracy	precision	recall	f1
Z + softmax + standardize (CCRE)	0.7530 ± 0.0000	0.7530 ± 0.0000	0.7530 ± 0.0000	0.7530 ± 0.0000
evidence only (4-D, no Z)	0.7530 ± 0.0000	0.7530 ± 0.0000	0.7530 ± 0.0000	0.7530 ± 0.000

- Observations on the ablation study
  1. **Standardization is the load-bearing step, not softmax:**
     - Z + softmax (no standardize) stays near vanilla ≈ 0.30 F1 — softmax probabilities live in [0, 1] with variance $\ll 1$, so K-means' Euclidean distance is dominated by the 20 unit-variance original columns and the evidence channel is effectively ignored.
     - Re-standardizing the softmax columns to unit variance is what lets K-means actually see them.
  2. **Raw Σ log(1+lift) works just as well as softmax + standardize.**
     - Both reach F1 ≈ 0.75 because both end up giving the evidence columns comparable per-row contrast across classes.
     - Softmax is the principled choice (probabilities sum to 1; clear semantics) but isn't strictly necessary for the gain — the standardization is.
  3. **The original 20 features barely contribute once evidence is present.**
     - evidence only (4-D) matches full CCRE at F1 ≈ 0.75. The four label-informed columns carry essentially all the discriminative signal K-means uses; the original features serve mainly as secondary geometry that doesn't change cluster assignments.
     - This also explains the 0.0000 seed std: with such a strong evidence channel, K-means converges to the same partition regardless of initialization.

5.5 Sensitivity to rule-pool size

How many class-consequent rules per class do we actually need? Sweep over a wide log range to see if the gain depends on the specific top-K choice (default = 60).



- **The method is robust across two orders of magnitude**

  F1 stays between **0.73 and 0.78** from top_per_class = 5 all the way to **500**.

  - **Smaller pools are slightly better**

    With only **5–10 rules per class**, F1 reaches ≈ **0.78** — modestly higher than the default of **60**. The highest-lift rules per class are very class-pure (lift ≈ **3.9** for `ram_q1 → class 0`); diluting them with lower-lift rules adds noisy evidence without adding signal.

  - **No risk of under-mining for reasonable thresholds**

Even at `top_per_class = 5`, the four classes get balanced representation (**20 rules total**), which is enough to discriminate **4 clusters**.

5.6 Summary

- **The problem:**

  Q4 established that vanilla K-means cannot recover price_range (F1 ≈ 0.29, ARI ≈ 0.006). Per Q4(e), ram carries almost all of the supervised discriminative signal, but it has no privileged direction in PCA space, so a purely Euclidean clusterer can't isolate it.

- **The proposal:**

  CCRE breaks the unsupervised ceiling by feeding K-means a 4-dimensional, label-informed evidence channel: FP-growth mines rules with price_range_X as the consequent, each rule that fires on a sample contributes log(1 + lift) to its class's score, and a softmax + standardization turns those scores into unit-variance columns concatenated to the 20 original standardized features (24-D total).

- **What worked:**

  F1 jumps from 0.29 → 0.75, with seed-to-seed std of 0.0000 — the augmented feature space leaves K-means essentially no degrees of freedom in initialization.

  The class-rule table makes the source of the gain interpretable: lift ≈ 3.9 for ram_q1 → class 0 and ram_q4 → class 3 shows FP-growth is surfacing exactly the RAM-quartile signal Q4(e) flagged as supervised-discriminative.

- **What the ablation isolated:**

  Standardizing the evidence columns is the load-bearing step; raw Σ log(1+lift) matches full CCRE, while softmax-without-standardize collapses back to chance because softmax probabilities live in [0, 1] and get drowned out by the 20 unit-variance original columns.

  evidence only (4-D) matches full CCRE — once the evidence channel is present, the original features contribute essentially nothing.

- **Robustness:**

  Sweeping top_per_class over two orders of magnitude (5 → 500) leaves F1 in a tight 0.73–0.78 band, so the method is not sensitive to its main hyperparameter.

References

Liu, B., Hsu, W., & Ma, Y. (1998, August). Integrating classification and association rule mining. In *Proceedings of the fourth international conference on knowledge discovery and data mining* (pp. 80-86).